CSC14003 Project 1 Report

FreeCell Solver

Course: CSC14003 - Introduction to Artificial Intelligence

Deliverable: Section 3.6 Report

Date: 2026-03-27

Team members

Member	Student ID	Role
TODO Member 1	TODO ID 1	TODO
TODO Member 2	TODO ID 2	TODO
TODO Member 3	TODO ID 3	TODO

Note: Team member names, student IDs, task distribution, and completion percentages must be filled in by the team before submission.

1. Introduction

This project implements a Python desktop FreeCell application with a tkinter GUI, a shared rules engine, and multiple search solvers. The current repository supports manual play, Microsoft numbered deals, random shuffled boards, built-in sample boards, background solver execution, live search metrics, and replay of either the final solution or the best-progress trace.

- Required algorithms present: BFS, DFS implemented as IDS, UCS, and A*.

- An additional Expert Solver exists in the repository but is not used as a substitute for the required algorithms in the main evaluation.

2. Project Planning and Task Distribution

Section 3.6 of the project brief requires each team member's responsibilities and completion percentage. The repository did not contain the actual team allocation data, so the table below is an explicit placeholder for the team to complete before submission.

Member	Student ID	Assigned tasks	Completion %	Notes
TODO Member 1	TODO ID 1	TODO	TODO	TODO
TODO Member 2	TODO ID 2	TODO	TODO	TODO
TODO Member 3	TODO ID 3	TODO	TODO	TODO

Brief example from page 5: if the group score is 9.0 and a student's completion percentage is 90%, the individual score is 9.0 x 90% = 8.1.

3. System Overview

The entry point is main.py, which launches gui.app.FreeCellApp. The codebase is organized into GUI, game-logic, solver, and test modules. The GUI handles drawing, user actions, solver controls, and replay. The shared game layer defines cards, immutable states, move generation, move application, Microsoft deals, and sample boards. The solver layer contains a shared BaseSolver plus BFS, DFS (IDS), UCS, A*, and an extra Expert Solver. Replay and live metrics flow through SolverResult, ProgressSnapshot, and the FreshQueue progress queue.

- GUI modules: gui/app.py, gui/progress_queue.py

- Game logic: game/card.py, game/state.py, game/moves.py, game/deal.py, game/ms_deals.py, game/samples.py

- Required solvers: solvers/bfs.py, solvers/dfs.py, solvers/ucs.py, solvers/astar.py

- Metrics and replay infrastructure: solvers/base.py and gui/app.py

- Tests: tests/*.py

4. Search Problem Formulation

The search state is game.state.GameState with eight cascades, four free cells, and four foundation counters. Legal actions are generated by game.moves.get_valid_moves() and include moves between cascades, free cells, and foundations, including sequence moves. apply_move() creates a new immutable GameState for every transition. The goal test is GameState.is_goal(), which checks that every foundation has reached rank 13. Repeated-state handling uses GameState.canonical_key(), which keeps the cascades exact, sorts free cells as a multiset, and includes the foundation tuple.

- BFS and IDS use implicit unit depth as their search depth measure.

- UCS uses a custom move cost: foundation = 0, free cell = 3, cascade = 1.

- A* uses unit move cost g plus a heuristic h.

- BFS suppresses duplicates through the parent dictionary; UCS and A* use best-cost tables; IDS uses path-based cycle checking only.

5. Algorithm Description

5.1 BFS

solvers/bfs.py implements BFS with a collections.deque frontier. States are popped in FIFO order using popleft(). Repeated states are avoided through the parent dictionary keyed by canonical_key(). The solver checks the goal before the loop and immediately after generating each child. Paths are reconstructed by BaseSolver._reconstruct(). BFS is complete over the explored state space but uses the most memory on difficult boards because it stores a large frontier.

5.2 DFS or IDS

The repository's DFS requirement is implemented as iterative deepening DFS in solvers/dfs.py, and the solver name is explicitly 'DFS (IDS)'. The search increases the depth limit from 1 upward and runs a recursive depth-limited search. It avoids cycles only along the current path using path_keys, so work is repeated across depth iterations. This keeps memory low but causes significantly higher runtime on harder boards.

5.3 UCS

solvers/ucs.py implements uniform-cost search with a heapq priority queue storing (g, counter, state). The actual path-cost function in the code assigns cost 0 to foundation moves, 3 to free-cell moves, and 1 to cascade moves. Repeated states are handled with a best_g table. UCS therefore optimizes the repository's custom move cost, not strictly the fewest number of moves.

5.4 A*

solvers/astar.py implements A* with a heapq frontier over f = g + h. The heuristic combines three terms: cards not yet on foundations, blocker depth for the next needed foundation cards, and a 0.5 penalty for occupied free cells. A direct repository counterexample shows the heuristic can exceed the true remaining cost, so the implementation should be described as heuristic best-first search with f = g + h but without a guaranteed optimality proof.

6. Experiments

The benchmark runner is report_tools/run_report_experiments.py. Every required algorithm was executed on the same fixed board set with the same settings: max nodes = 5,000, use_auto_moves = False, and 3 trials per algorithm-board pair. Medians are reported. The performance metrics are taken directly from BaseSolver.solve(), which measures elapsed search time and peak memory via tracemalloc and returns expanded nodes and solution length through SolverResult.

Item	Value
Python version	3.10.11
Python implementation	CPython
Platform	Windows-10-10.0.22631-SP0
Processor string	Intel64 Family 6 Model 167 Stepping 1, GenuineIntel

Item	Value
Machine	AMD64
Trials per case	3
Equal node cap	5000
use_auto_moves	False

Benchmark set	Board	Purpose
Core	near_goal	Single-move finish state used in solver tests
Core	easy_demo	Built-in sample board from the repository
Core	two_suit_finish	Hand-crafted 4-move finish state
Core	medium_demo	Harder built-in sample board
Stress	ms_deal_1	Full Microsoft deal #1 under the same node cap

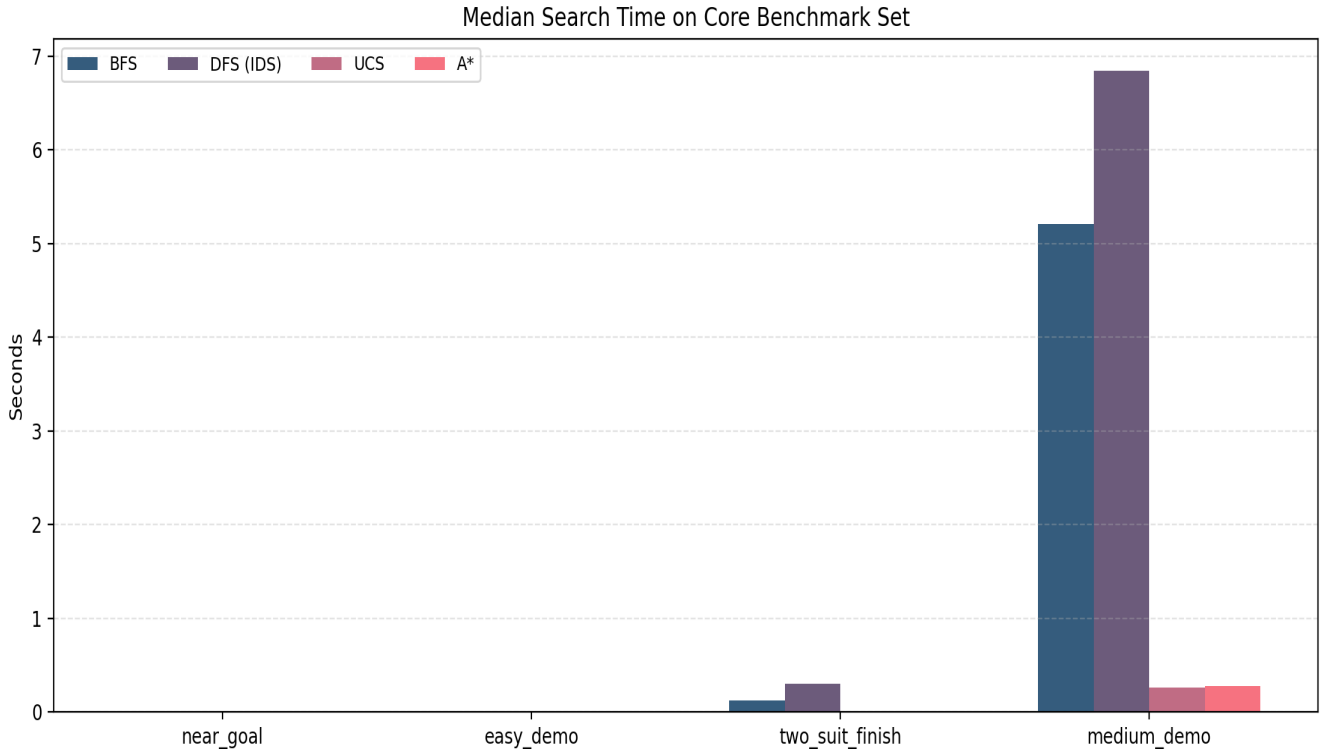
7. Experimental Results

Board	Algorithm	Solved	Time (s)	Peak KB	Expanded	Solution	Status
easy_demo	A*	Yes	0.000318	5.73	2	1	Solved
easy_demo	BFS	Yes	9.1e-05	2.469	1	1	Solved
easy_demo	DFS (IDS)	Yes	8.7e-05	2.375	1	1	Solved
easy_demo	UCS	Yes	0.000303	5.625	2	1	Solved
medium_demo	A*	Yes	0.274063	4687.719	256	12	Solved
medium_demo	BFS	No	5.207949	43092.852	5000	0	Node limit reached
medium_demo	DFS (IDS)	No	6.843405	25.625	5000	0	Node limit reached
medium_demo	UCS	Yes	0.26532	4626.531	256	12	Solved
near_goal	A*	Yes	0.000329	5.688	2	1	Solved
near_goal	BFS	Yes	9.8e-05	2.469	1	1	Solved
near_goal	DFS (IDS)	Yes	9.8e-05	2.789	1	1	Solved
near_goal	UCS	Yes	0.000311	5.625	2	1	Solved
two_suit_finish	A*	Yes	0.003435	42.844	9	4	Solved
two_suit_finish	BFS	Yes	0.127405	588.949	194	4	Solved
two_suit_finish	DFS (IDS)	Yes	0.301666	11.229	371	4	Solved
two_suit_finish	UCS	Yes	0.00429	42.742	9	4	Solved

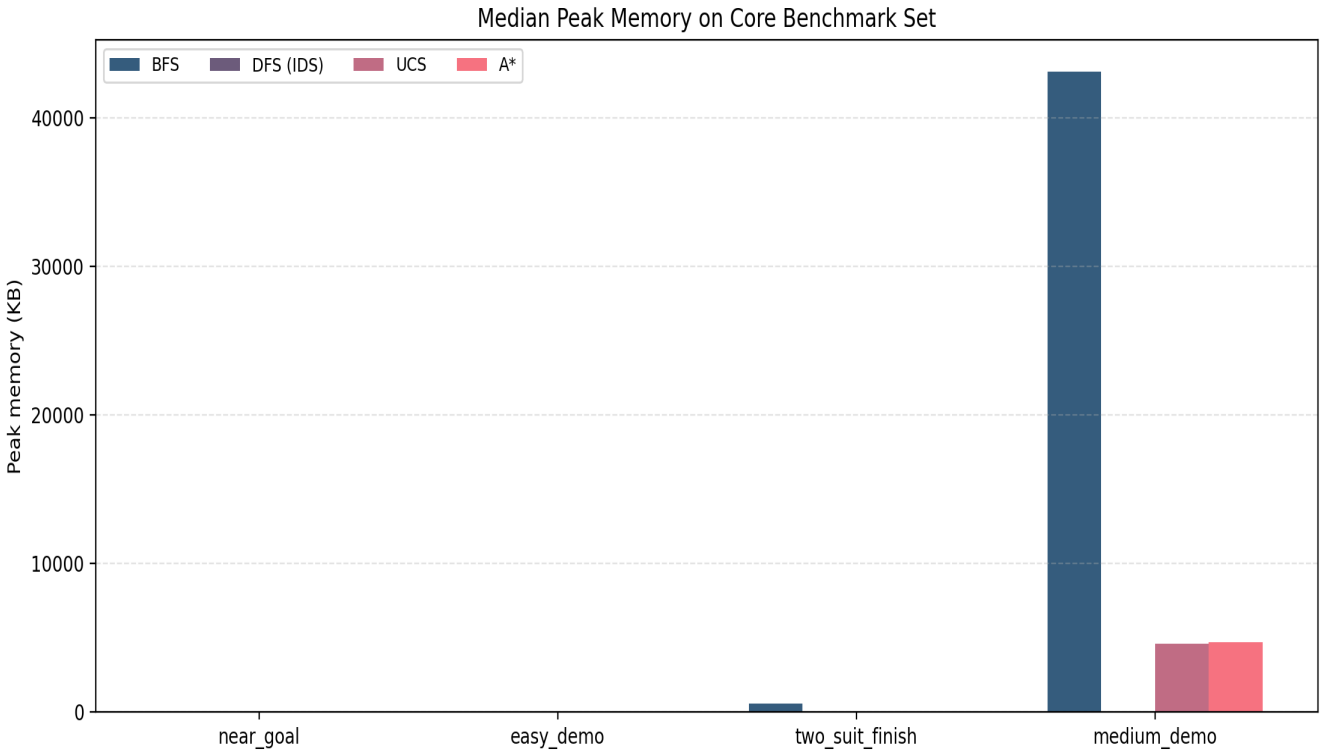
Board	Algorithm	Solved	Time (s)	Peak KB	Expanded	Solution	Status
ms_deal_1	A*	No	2.882311	19465.168	5000	0	Node limit reached
ms_deal_1	BFS	No	2.063527	12224.195	5000	0	Node limit reached
ms_deal_1	DFS (IDS)	No	3.040895	13.664	5000	0	Node limit reached
ms_deal_1	UCS	No	3.425165	25728.516	5000	0	Node limit reached

Key observation: BFS and IDS solve the small finish states but both exhaust the equal 5,000-node budget on medium_demo. UCS and A* solve all core boards, but all four required algorithms fail on the full Microsoft stress case under the same budget.

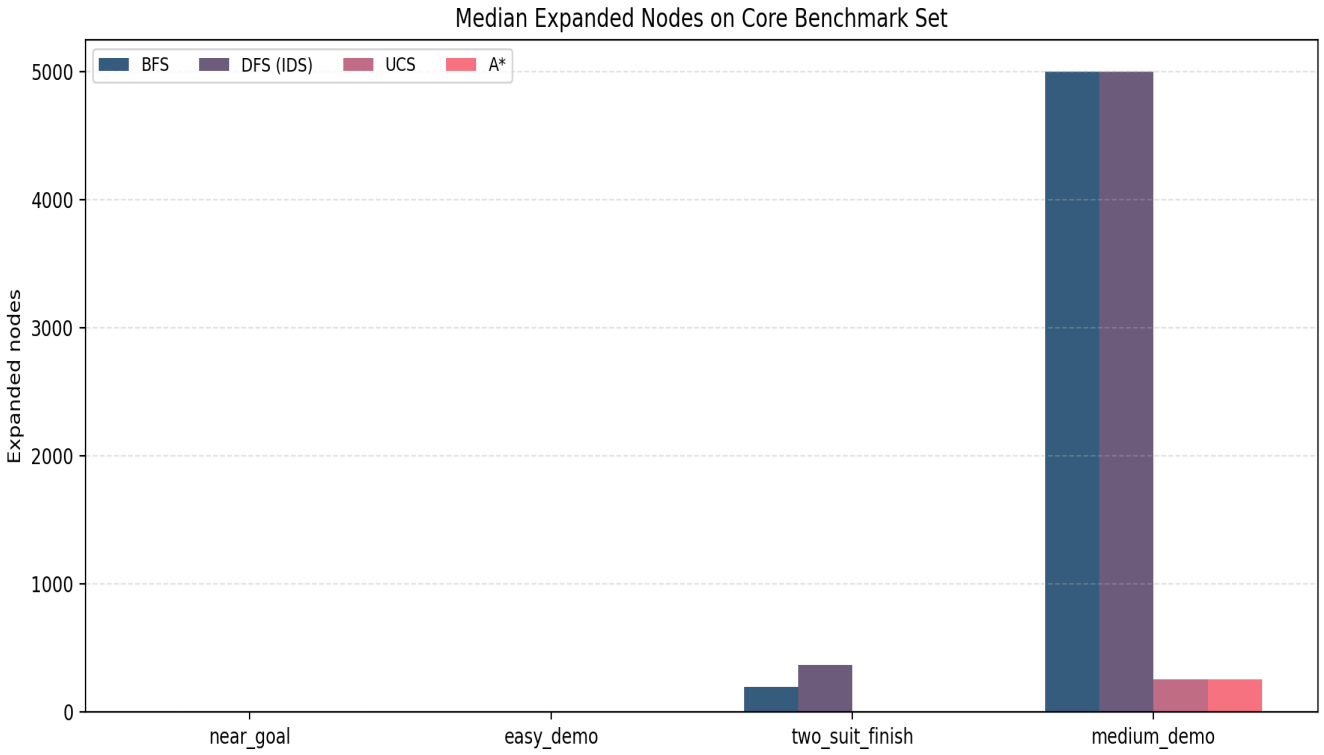
Search time comparison



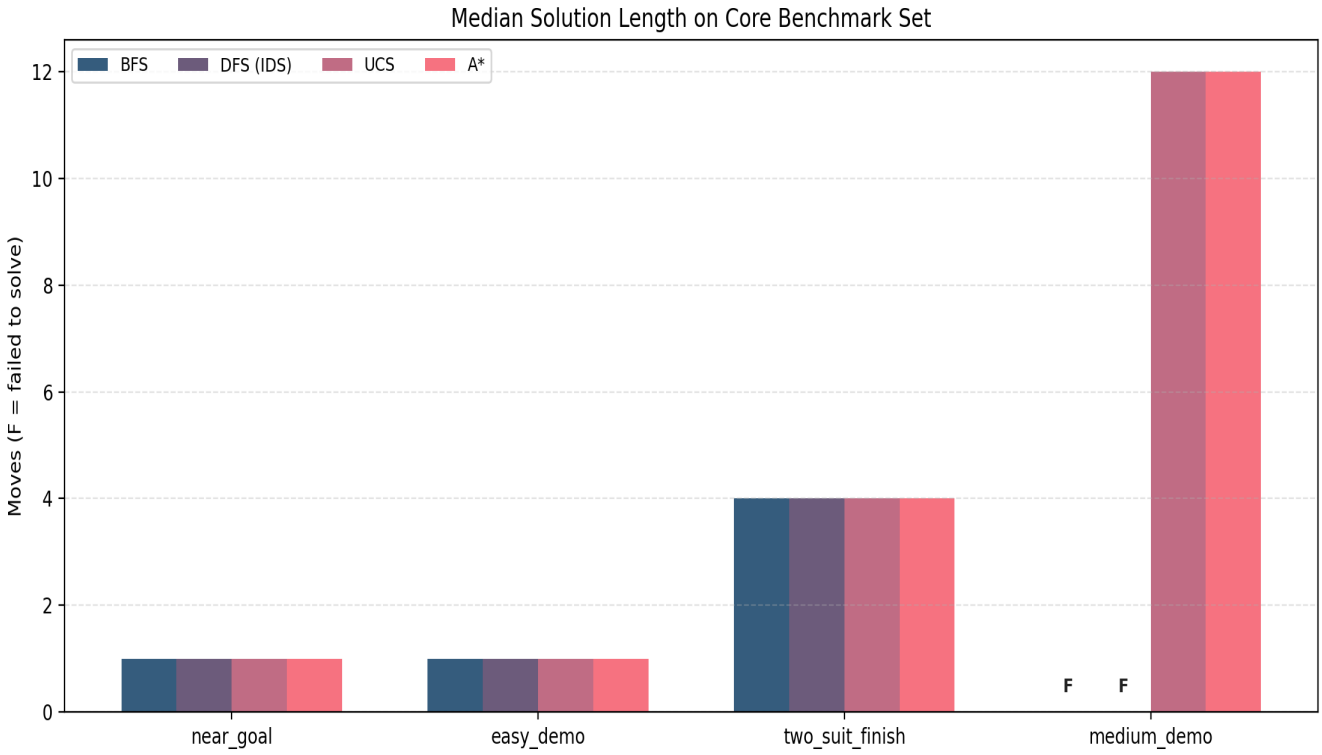
Peak memory comparison



Expanded nodes comparison



Solution length comparison



In the solution-length chart, F marks algorithms that failed to solve the board within the equal node cap.

8. Discussion

Among the four required algorithms, UCS and A* are the most practical in the current repository. They solve every core benchmark board under the equal 5,000-node budget, while BFS and IDS both fail on medium_demo. BFS uses the most memory because it maintains a large frontier. IDS keeps memory very low but pays with repeated work and the slowest runtime on the harder sample board. UCS and A* are close on the tested core set; both solve medium_demo in about 0.27 seconds and expand 256 nodes. A* should still be described carefully because the current heuristic is not formally admissible.

- BFS: simple and complete, but memory-heavy.

- DFS (IDS): memory-light, but expensive in runtime because of repeated iterations.

- UCS: strong under the repository's custom move cost.

- A*: strong practical performance on the tested core set, but without a valid optimality guarantee under the current heuristic.

- Expert Solver exists as an extra implementation note, but it is outside the required four-algorithm comparison.

9. Conclusion

The FreeCell Solver repository satisfies the main structural requirement of implementing handwritten search algorithms inside a playable Python GUI. The shared rules engine, immutable state model, and solver metrics make the codebase suitable for reproducible evaluation. Under the equal 5,000-node budget used in this report, BFS and IDS are limited to small finish states, while UCS and A* also solve the harder medium_demo sample board. In the current implementation, UCS and A* are the most practical among the four required

algorithms.

10. References

- 1. CSC14003 - Project 1.pdf, Section 3.6 'Report', page 5. Local course brief PDF found at C:\MEGA\co so ttnt\CSC14003 - Project 1.pdf.

- 2. Repository source code used as primary implementation evidence: main.py, gui/app.py, gui/progress_queue.py, game/state.py, game/moves.py, game/deal.py, game/ms_deals.py, solvers/base.py, solvers/bfs.py, solvers/dfs.py, solvers/ucs.py, solvers/astar.py.

- 3. Repository test modules used to validate current behavior: tests/test_app_flow.py, tests/test_solver_behavior.py, tests/test_solver_replay.py, tests/test_replay_controls.py, tests/test_ms_deals.py, tests/test_layout_and_rules.py.

11. Appendix

11.1 Generated experiment artifacts

- report_data/experiment_results_raw.csv

- report_data/experiment_results_summary.csv

- report_data/experiment_environment.json

- report_assets/chart_search_time.png

- report_assets/chart_peak_memory.png

- report_assets/chart_expanded_nodes.png

- report_assets/chart_solution_length.png

11.2 Commands used

- python report_tools/run_report_experiments.py

- python -m unittest discover -s tests

- python main.py

11.3 Generative AI usage

Generative AI was used in this report-generation session to audit the repository, extract implementation details, run reproducible experiments, summarize the results, generate charts, draft the report text, and build the final PDF. All numeric values in the report come from actual CSV outputs generated from the current repository, and the final text was checked manually against those artifacts.

11.4 Prompt record

Prompt / instruction source	What it was used for
Expert Solver patch request	Added the Expert Solver and related tests earlier in this session.
README / requirements audit request	Produced up-to-date run instructions and dependency notes earlier in this session.
Section 3.6 report generation request	Used for this report-generation run.
TODO: add other team-used prompts	Fill in before submission if applicable.